

多模态

- 类型1：输入与输出模态不同
- 类型2：多模态输入
- 类型3：多模态输出

GPT-4V

(2) Prompt:
Describe the pointed region in the image.

Method	Validataion set								Test set							
	in.		near.		out.		overall		in.		near.		out.		overall	
	C	S	C	S	C	S	C	S	C	S	C	S	C	S	C	S
OSCAR	85.4	11.9	84.0	11.7	80.3	10.0	83.4	11.4	84.8	12.1	82.1	11.5	73.8	9.7	80.9	11.3
Human	84.4	14.3	85.0	14.3	95.7	14.0	87.1	14.2	80.6	15.0	84.6	14.7	91.6	14.2	85.3	14.6
VIVO	92.2	12.9	87.8	12.6	87.5	11.5	88.3	12.4	89.0	12.9	87.8	12.6	80.1	11.1	86.6	12.4
VinVL	103.7	13.7	95.6	13.4	83.8	11.9	94.3	13.1	98.0	13.6	95.2	13.4	78.0	11.5	92.5	13.1
UFO	103.9	14.5	95.5	13.8	83.5	12.3	94.3	13.6	98.9	14.3	94.7	13.9	77.9	12.1	92.3	13.6
mPLUG	-	-	-	-	-	-	114.8	14.8	-	-	-	-	-	-	-	-
SimVLM	113.7	-	110.9	-	115.2	-	115.2	-	113.7	-	110.9	-	115.2	-	115.2	-
LEMON	118.0	15.4	116.3	15.1	120.2	14.5	117.3	15.0	112.8	15.2	115.5	15.1	110.1	13.7	114.3	14.9
UniversalCap	123.2	15.0	121.5	15.3	123.4	14.4	122.1	15.0	118.9	15.4	120.6	15.3	114.3	14.1	119.3	15.1
CoCa	-	-	-	-	-	-	122.4	15.5	-	-	-	-	-	-	120.6	15.5
GIT _B	100.7	13.8	97.7	13.5	89.6	12.5	96.6	13.4	-	-	-	-	-	-	-	-
GIT _L	107.7	14.9	107.8	14.5	102.5	13.7	106.9	14.4	-	-	-	-	-	-	-	-
GIT	129.8	16.3	124.1	16.0	127.1	15.7	125.5	16.0	122.4	16.2	123.9	16.0	122.0	15.7	123.4	15.9
GIT2	126.9	16.1	125.8	16.2	130.6	15.8	126.9	16.1	124.2	16.4	125.5	16.1	122.3	15.6	124.8	16.1

GPT-4V:
The highlighted red region in the image is a table cell that contains a numerical value of 122.3. It is located in the "Test set" section of the table, under the "out" column and the "GIT2" row. This value represents the overall performance metric for the GIT2 method on the test set, in the "out" category.

Prompt:
Localize each person in the image using bounding box. What is the image size of the input image?



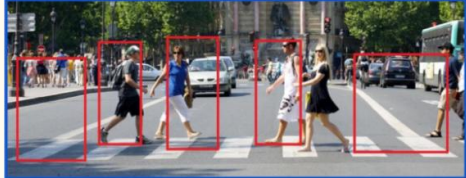
GPT-4V:
The image size of the input image is 1000 x 333 pixels.

Bounding boxes for each person:

- Person 1 (leftmost): (x1, y1) = (22, 108), (x2, y2) = (170, 300)
- Person 2: (x1, y1) = (200, 80), (x2, y2) = (290, 270)
- Person 3: (x1, y1) = (350, 70), (x2, y2) = (460, 280)
- Person 4: (x1, y1) = (540, 80), (x2, y2) = (640, 270)
- Person 5 (rightmost): (x1, y1) = (760, 100), (x2, y2) = (920, 290)

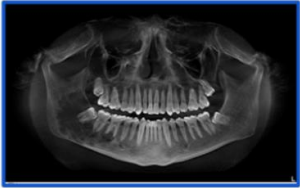
Note: These bounding box coordinates are approximate and may not be exact.

Visualization of Detected Bbox:



必看：微软166页论文解读 GPT-4V 全文
翻译《多模态的新时代》The Dawn of
LMMs: Preliminary Explorations with
GPT-4V(ision) - 知乎

Prompt:
Which wisdom tooth is hardest to remove?



Note: Same query image as shown above, resized to save space

GPT-4V:
The lower wisdom teeth, especially those that are impacted, are typically considered the hardest to remove. This is because the lower jawbone is denser than the upper jawbone, and the impacted wisdom teeth may be more difficult to access. Additionally, the roots of the lower wisdom teeth are often longer and more curved than those of the upper wisdom teeth, making them more challenging to remove.

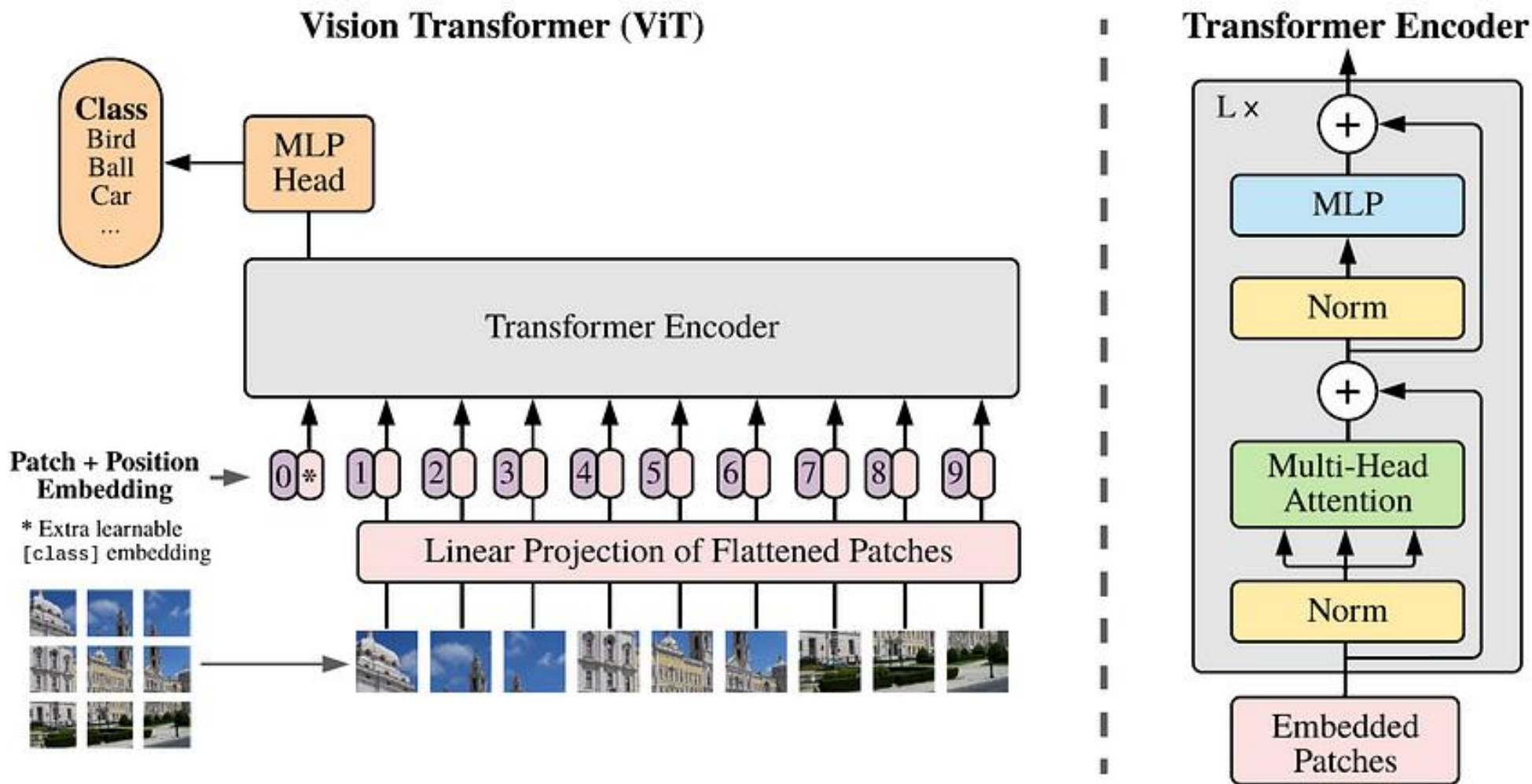
Prompt:
Count the number of apples in the image.



GPT-4V:
There are 7 apples in the image.

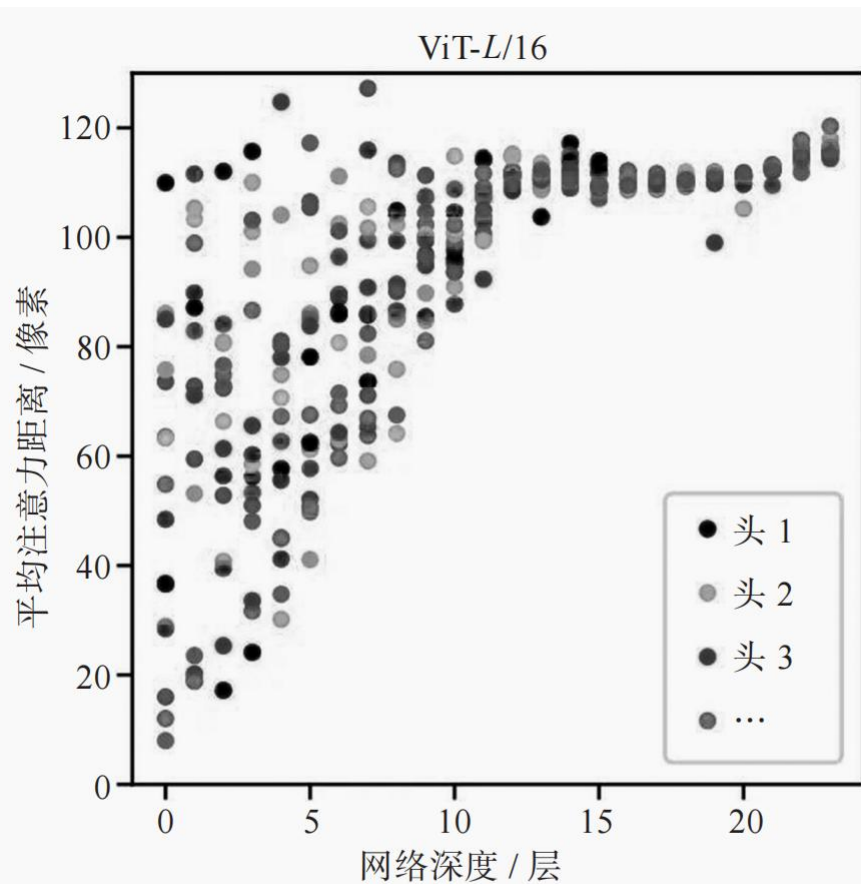
ViT

输入图像的分辨率为 224×224 像素，切分后的图像小块的分辨率为 16×16 像素，此时的序列长度为 $(224 / 16) \times (224 / 16) = 196$ ，每个元素的维度就是 $16 \times 16 \times 3 = 768$ ，这对于一般的机器来说是可以接受的训练 Transformer 模型的长度。



ViT

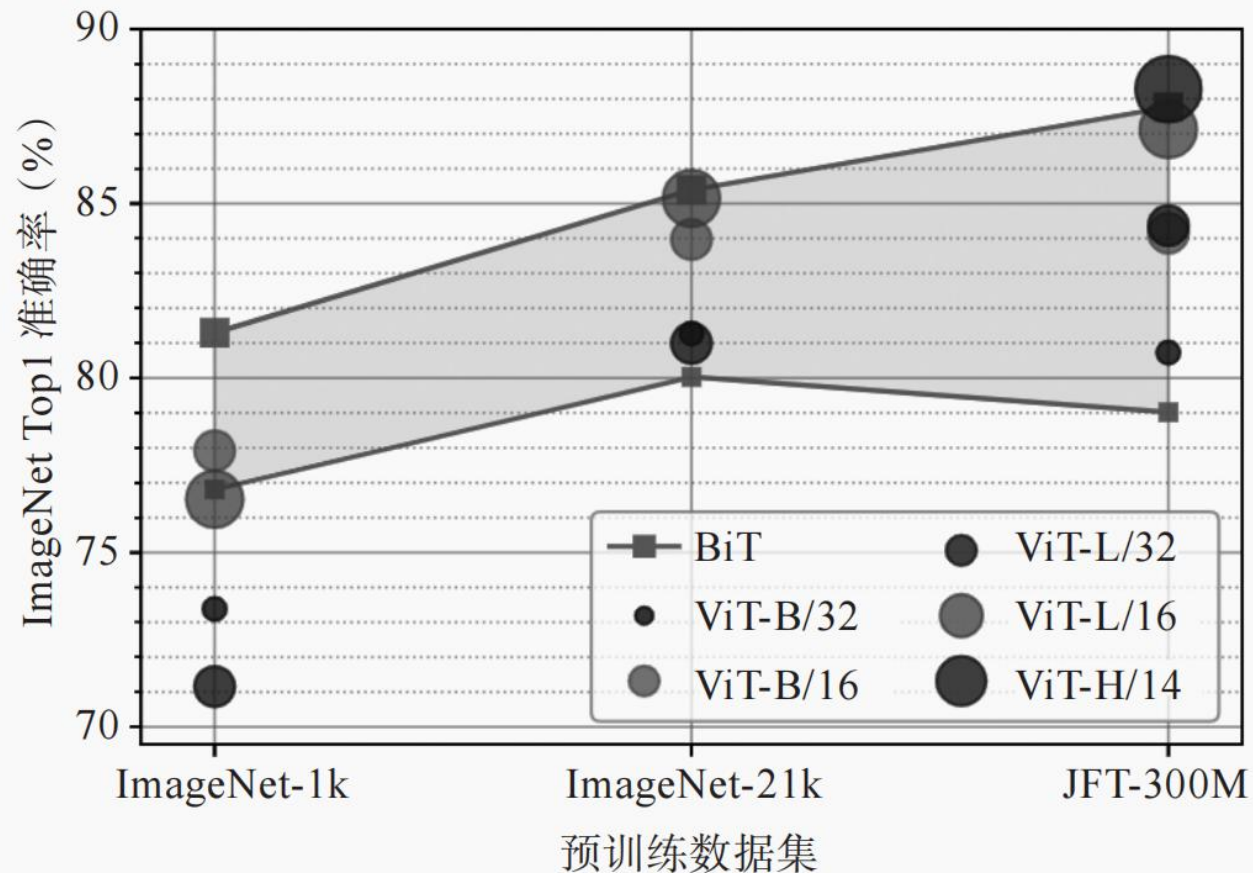
编码器的每一层会重复 L 次，每次在注意力层都对输入序列中的每个图像块进行上下文相关的特征编码，捕获图像块之间丰富的空间关系。在图像上应用注意力机制使得 ViT 能够整合整个图像的信息，如图所示，根据多头注意力权重计算图像空间中信息整合的平均距离，类似于卷积神经网络中的感受野大小。可以发现，在刚开始时，有的注意力头之间的距离很近，而有的注意力头之间的距离很远，这说明一开始模型可以关注全局信息。随着网络深度的增加，多头之间的距离在变大，这说明网络已经不再通过邻近像素点获取特征，而是已经学习到了高层的语义信息。



ViT 中多头注意力权重对应的空间信息整合距离随网络深度变化示意

大模型架构

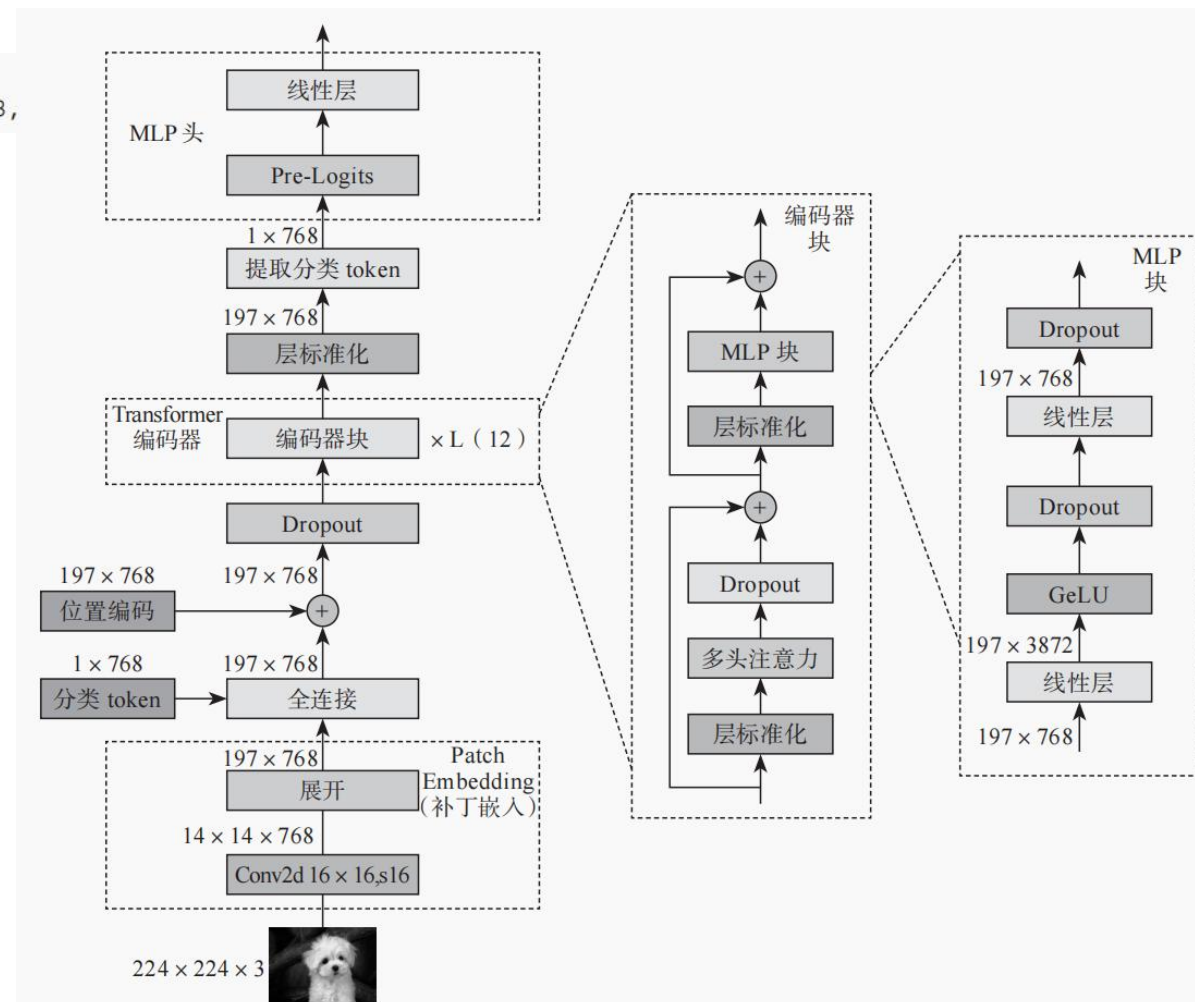
混合架构的模型往往会使用一个小的卷积神经网络作为特征提取器，从原始图像中提取特征图，通常是一些局部特征，如边缘、纹理等，然后将这些特征图输入 Transformer 模型中进行处理，提取全局依赖关系，如物体的位置、大小等。



ViT 与卷积神经网络在不同预训练数据集上的 ImageNet Top1 准确率对比

```
class ViT(nn.Module):
    def __init__(self, image_size=224, patch_size=16, num_classes=1000, dim=768,
        depth=12, heads=12, mlp_dim=3872):
        super(ViT, self).__init__()
        self.image_size = image_size # 输入图像的大小
        self.patch_size = patch_size # 切分图像的块大小
        self.num_patches = (image_size // patch_size) ** 2 # 计算总的图像块数量
        self.patch_dim = 3 * patch_size ** 2 # 每个图像块的维度
        # 利用卷积层将图像切分为多个图像块, 并将每个图像块投影到 dim 维空间
        self.conv = nn.Conv2d(3, dim, kernel_size=patch_size, stride=patch_size)
        self.cls_token = nn.Parameter(torch.randn(1, 1, dim)) # CLS token
        # 使用 Transformer 对图像块进行编码
        self.transformer_encoder = nn.TransformerEncoder(nn.
            TransformerEncoderLayer(d_model=dim, nhead=heads, dim_
            feedforward=mlp_dim), num_layers=depth)
        # 添加 LayerNorm 层
        self.layer_norm = nn.LayerNorm(dim)
        # 全连接层, 用于分类任务
        self.fc = nn.Linear(dim, num_classes)

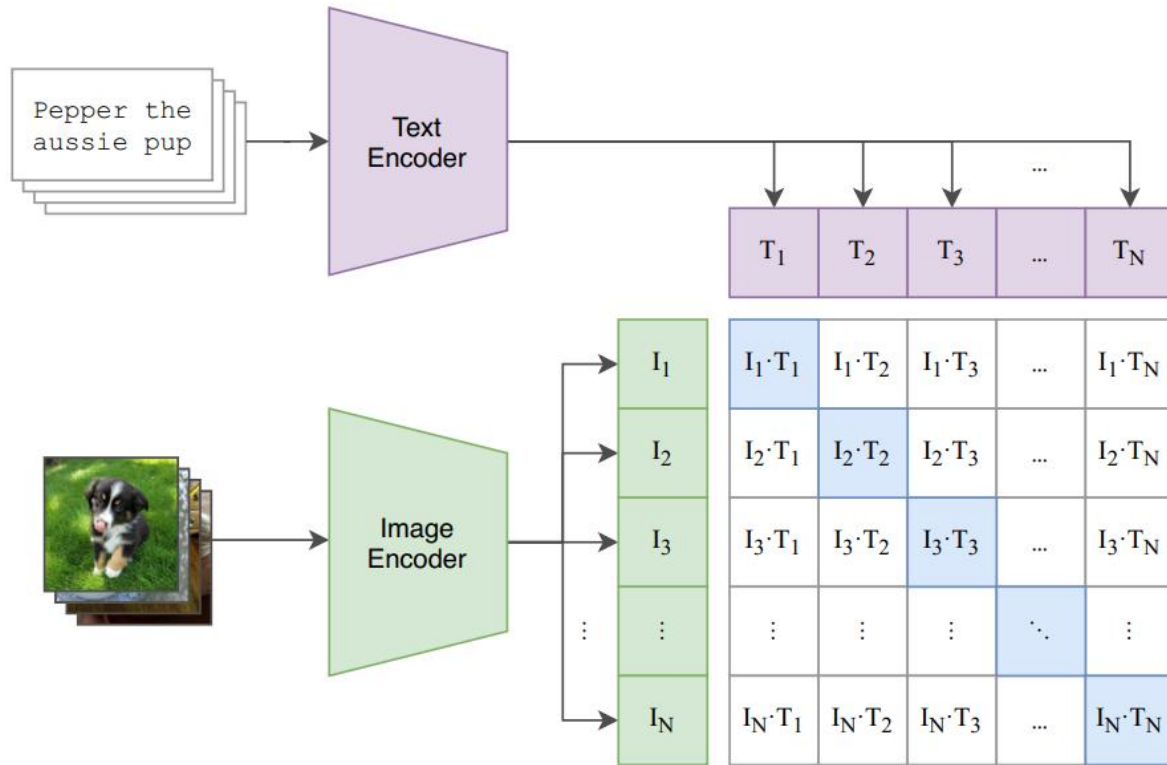
    def forward(self, x):
        # 图像通过卷积层进行切分和线性投影
        x = self.conv(x)
        # 对卷积层的输出进行变形 (reshape), 以符合 Transformer 的输入要求
        x = x.flatten(2).transpose(1, 2)
        # 在序列的开始位置添加 CLS token
        cls_tokens = self.cls_token.expand(x.shape[0], -1, -1) # 对 CLS token 进
            行复制以匹配批量大小
        x = torch.cat((cls_tokens, x), dim=1)
        # 图像块通过 Transformer 进行编码
        x = self.transformer_encoder(x)
        # 取出 CLS token 的表征用于分类
        x = x[:, 0]
        x = self.layer_norm(x)
        # 通过全连接层进行分类
        x = self.fc(x)
        return x
```



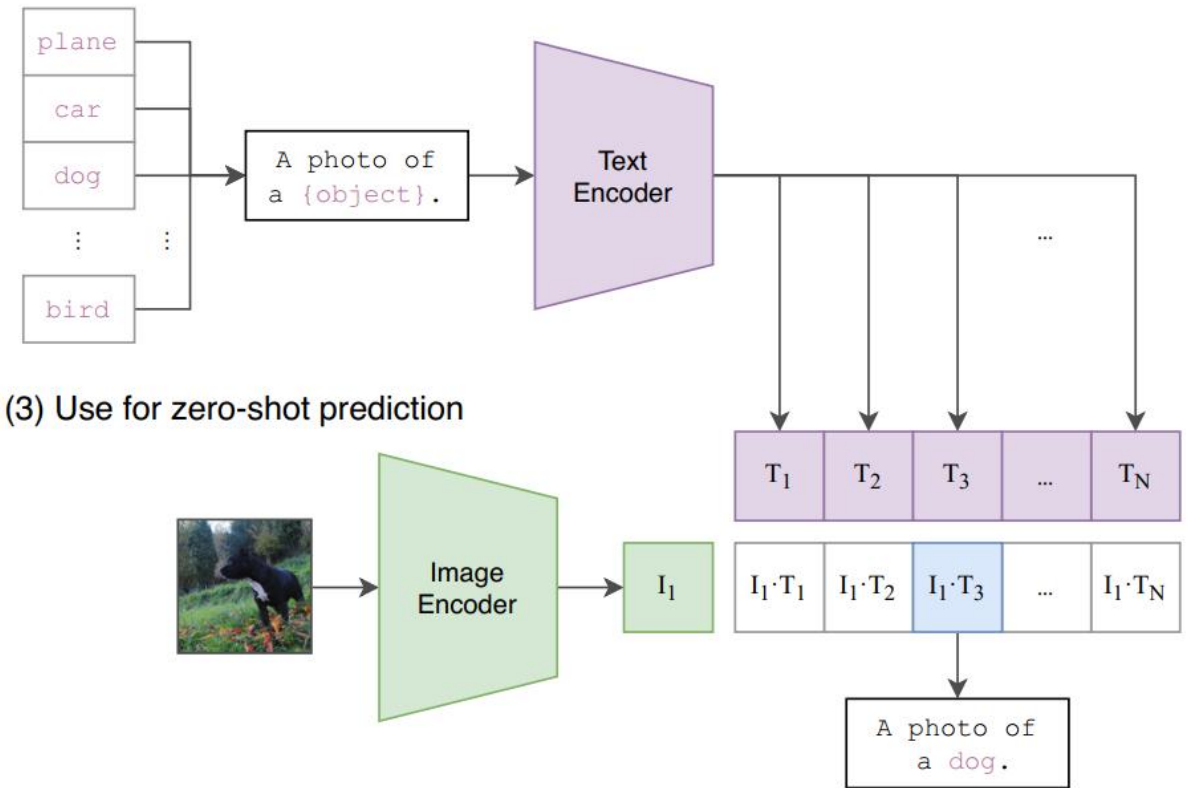
Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

□ CLIP (Contrastive Language-Image Pre-training)

(1) Contrastive pre-training



(2) Create dataset classifier from label text

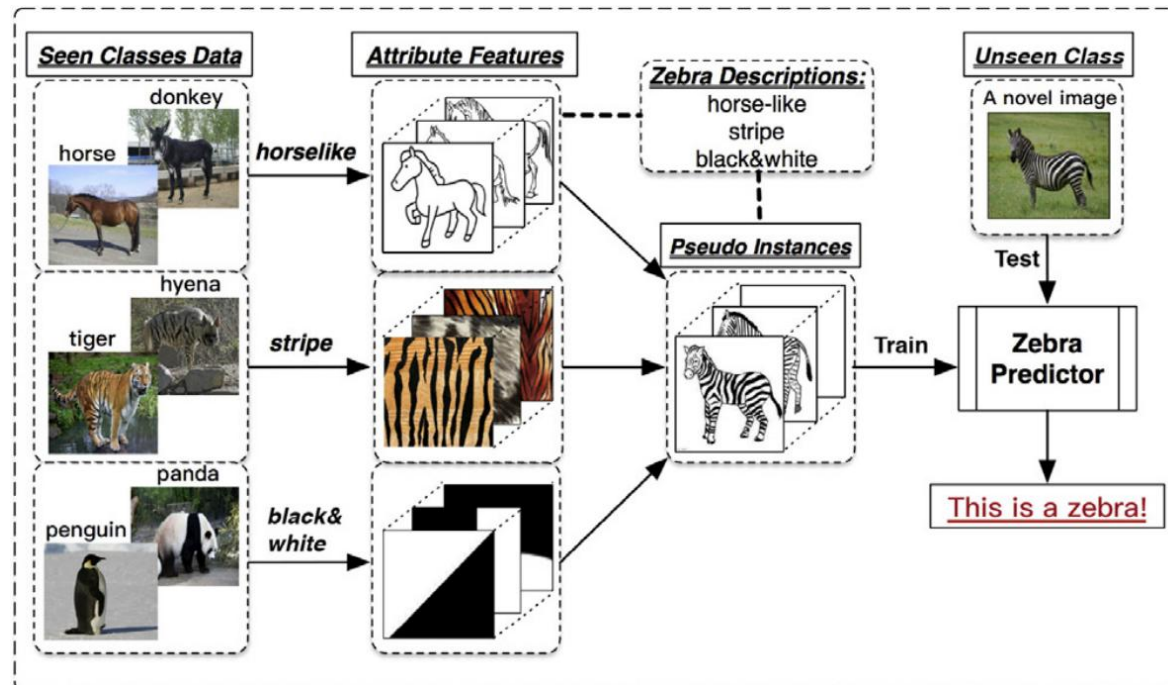
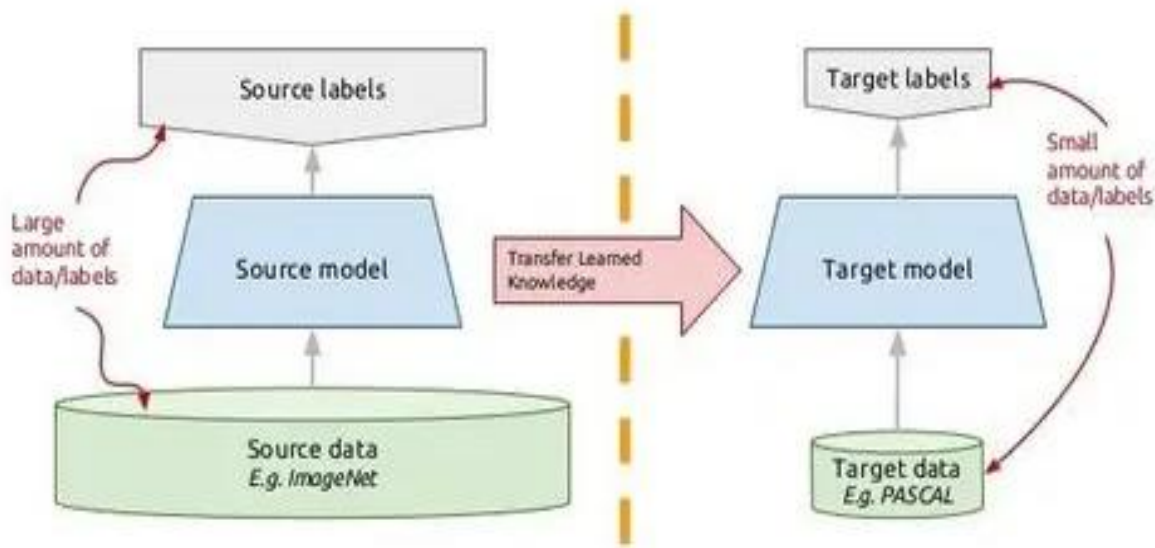


(3) Use for zero-shot prediction

CLIP 动机

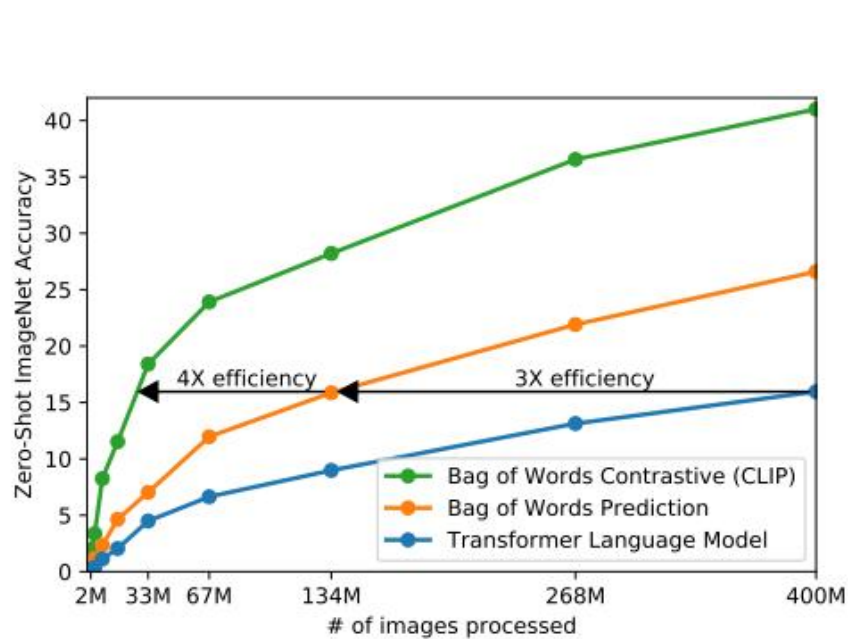
CLIP: **C**ontrastive Language–Image Pre-training

迁移学习与零样本学习:

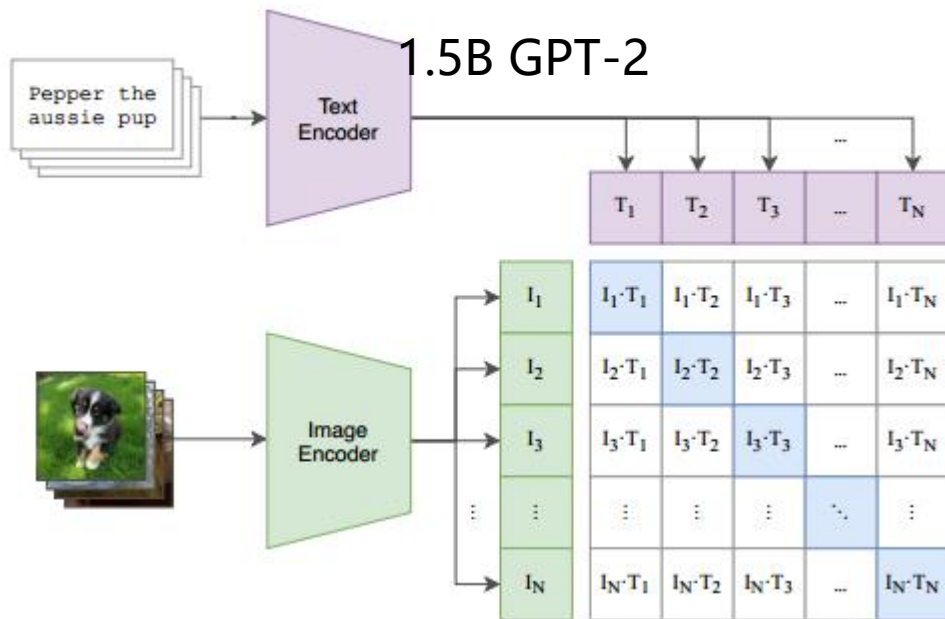


- 当前迁移学习的流行范式仍需要少量任务特定样本对模型进行微调，作者希望利用**自然语言**做监督信号实现**零样本学习**。
- 之前的工作受限于**模型的发展**和**数据的规模**没有取得很好的效果，而作者采用了**先进的 Transformer 结构**，并且从互联网爬取了包含**4亿图像-文本对**的数据集WIT。

CLIP 模型训练 256个V100+12天时间



(1) Contrastive pre-training



```
# image_encoder - ResNet or Vision Transformer
# text_encoder - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l] - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t - learned temperature parameter
```

```
# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T) #[n, d_t]
```

```
# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)
```

```
# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)
```

```
# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss = (loss_i + loss_t)/2
```

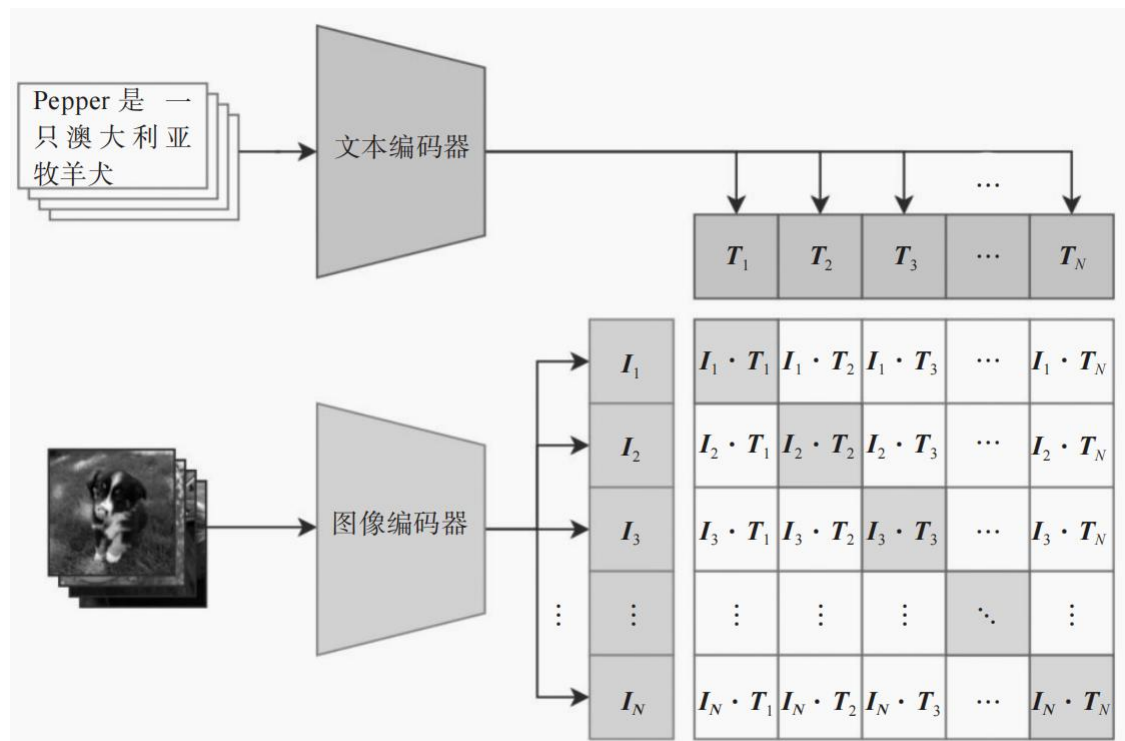
Figure 3. Numpy-like pseudocode for the core of an implementation of CLIP.

- 预测性任务训练效率低，**对比性任务训练效率高**。
- 模型通过海量图像文本对实现图像和文本特征在**向量空间上的对齐**。

CLIP

输入的图片是一只狗，而对应的文字描述是 “Pepper 是一只澳大利亚牧羊犬”。

N 张图片的描述句子会通过文本编码器得到 N 个特征向量，然后 N 张图片也会通过图像编码器得到 N 个特征向量，这里的图像编码器可以任意选择，论文中使用了 ResNet 和 ViT 两种。CLIP 就是根据 N 个图像特征向量和 N 个文本特征向量做对比学习，一个配对的“图像 - 文本”对就是一个正样本，而其他的都是负样本。因此，在一个矩阵对应关系上，对角线上的 N 个元素都是正样本，非对角线的 $N^2 - N$ 个元素都是负样本。



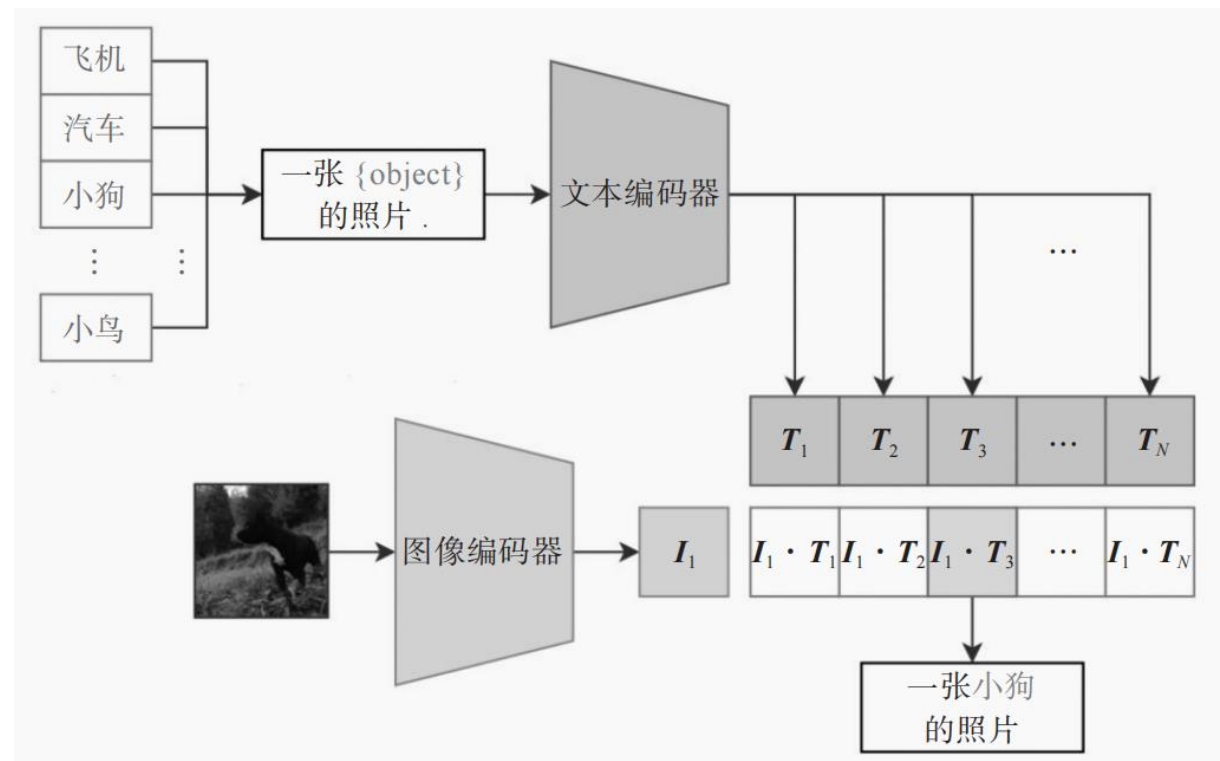
CLIP 模型的“图像 - 文本”编码对比学习示意图

CLIP

给定一张图片，经过图像编码器得到 1 个图像特征，然后将可能的 N 个标签通过提示词工程得到 N 个文本描述，再经过文本编码器得到 N 个文本特征，最后用 1 个图像特征和 N 个文本特征计算余弦相似度，再通过一个 softmax 函数，得到最终的分概率分布。

	Dataset Examples	ImageNet Zero-Shot		
		ResNet101	CLIP	Δ Score
ImageNet		76.2	76.2	0%
ImageNetV2		64.3	70.1	+5.8%
ImageNet-R		37.7	88.9	+51.2%
ObjectNet		32.6	72.3	+39.7%
ImageNet Sketch		25.2	60.2	+35.0%
ImageNet-A		2.7	77.1	+74.4%

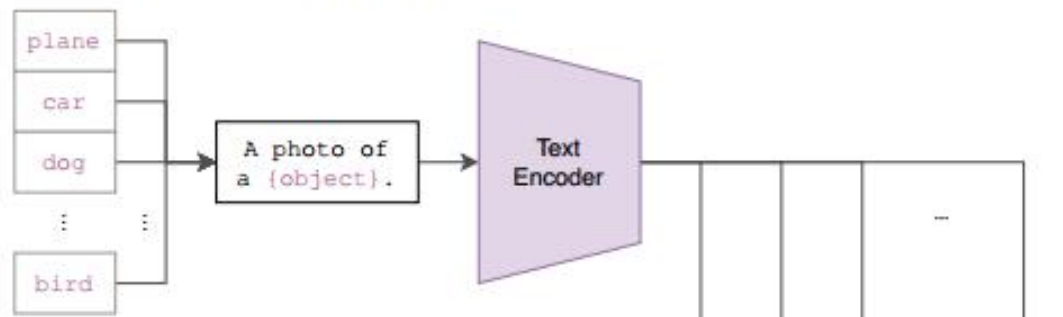
CLIP 模型在不同 ImageNet 衍生数据集上的泛化能力对比



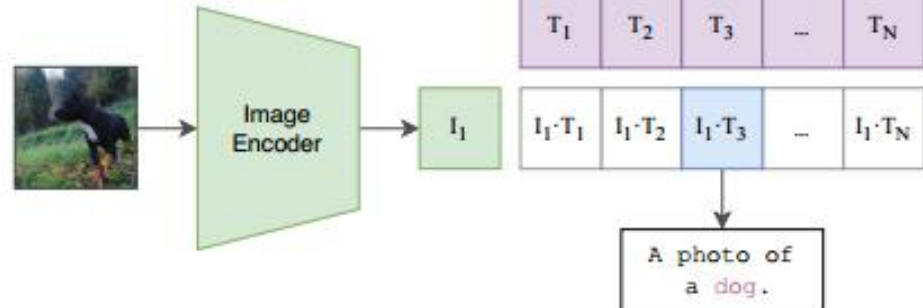
CLIP 模型推测

256个V100+12天时间

(2) Create dataset classifier from label text



(3) Use for zero-shot prediction



prompt learning:

- 采用原因

- 一词多义
- 训练文本基本是句子
- 提供任务先验。

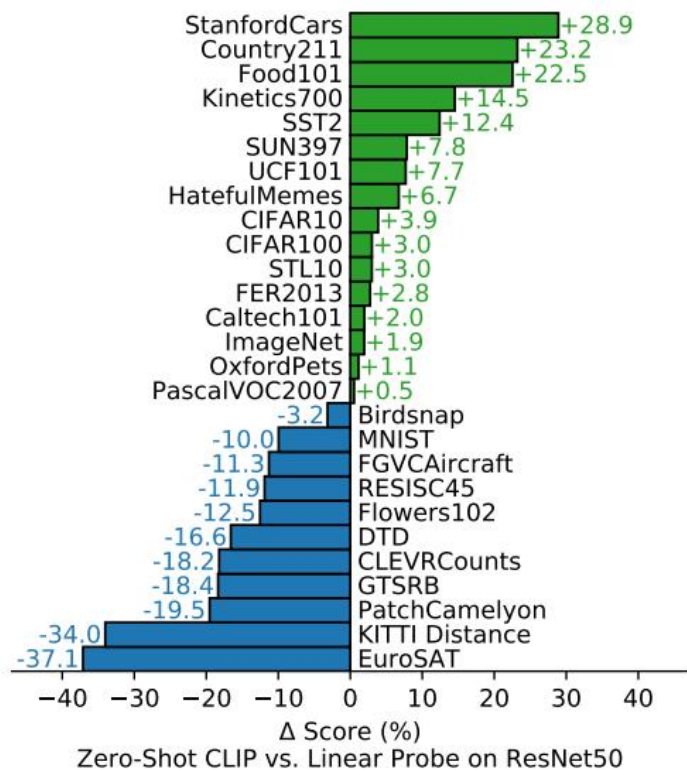
- 做法

- 集成多个prompt
- 嵌入空间取平均
- 作者在ImageNet集成了80个prompt, 提升5%。

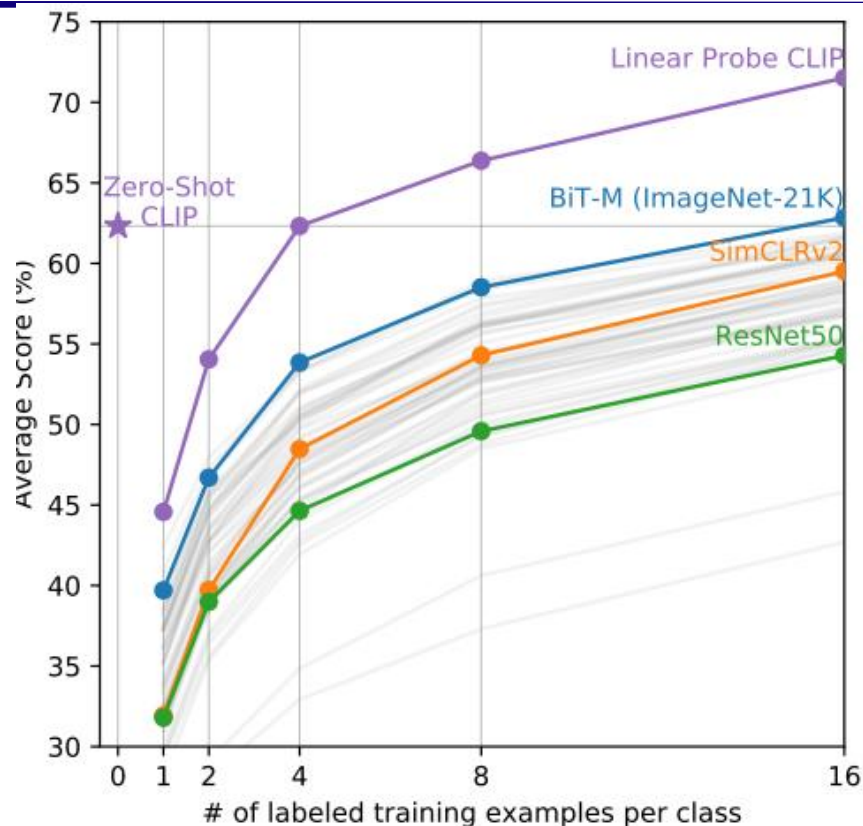
- 使用提示工程。

- 最大相似度即为预测类。

CLIP 实验



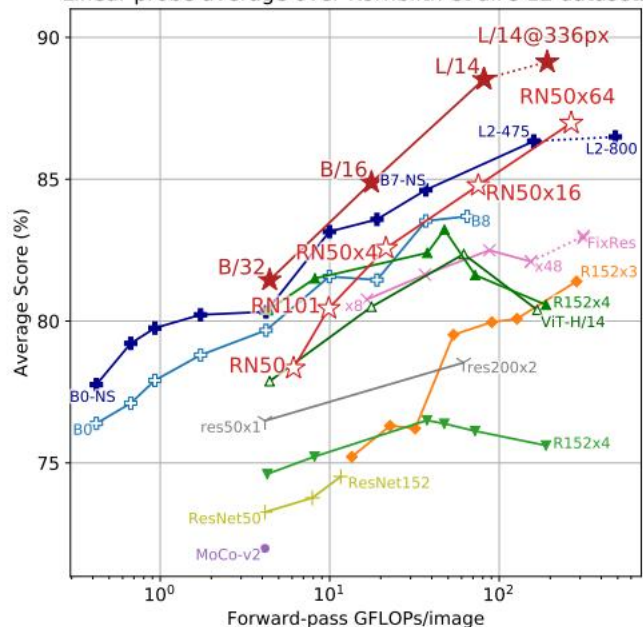
- 在27个数据集上将**CLIP零样本性能**与预训练+微调的有监督resnet对比。结果显示CLIP对物体分类问题表现好，对纹理分类DTD、计数CLEVRCOUNTS等表现不佳。



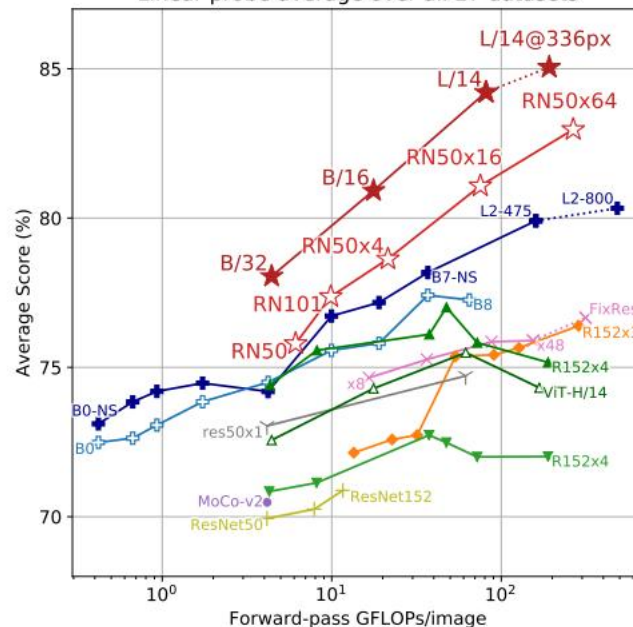
- 在20个数据集进行**few-shot 实验**。取每个模型图像特征提取器冻住微调全连接层，展现了CLIP的FSL能力。样本较少的微调不如zero-shot。

CLIP 实验

Linear probe average over Kornblith et al.'s 12 datasets



Linear probe average over all 27 datasets



	Dataset Examples						ImageNet ResNet101	Zero-Shot CLIP	Δ Score
ImageNet							76.2	76.2	0%
ImageNetV2							64.3	70.1	+5.8%
ImageNet-R							37.7	88.9	+51.2%
ObjectNet							32.6	72.3	+39.7%
ImageNet Sketch							25.2	60.2	+35.0%
ImageNet-A							2.7	77.1	+74.4%

- 在多个数据集上linear prob测试表示学习的性能。展现了CLIP强大的表示学习能力。

- 展现了稳健性，泛化能力。

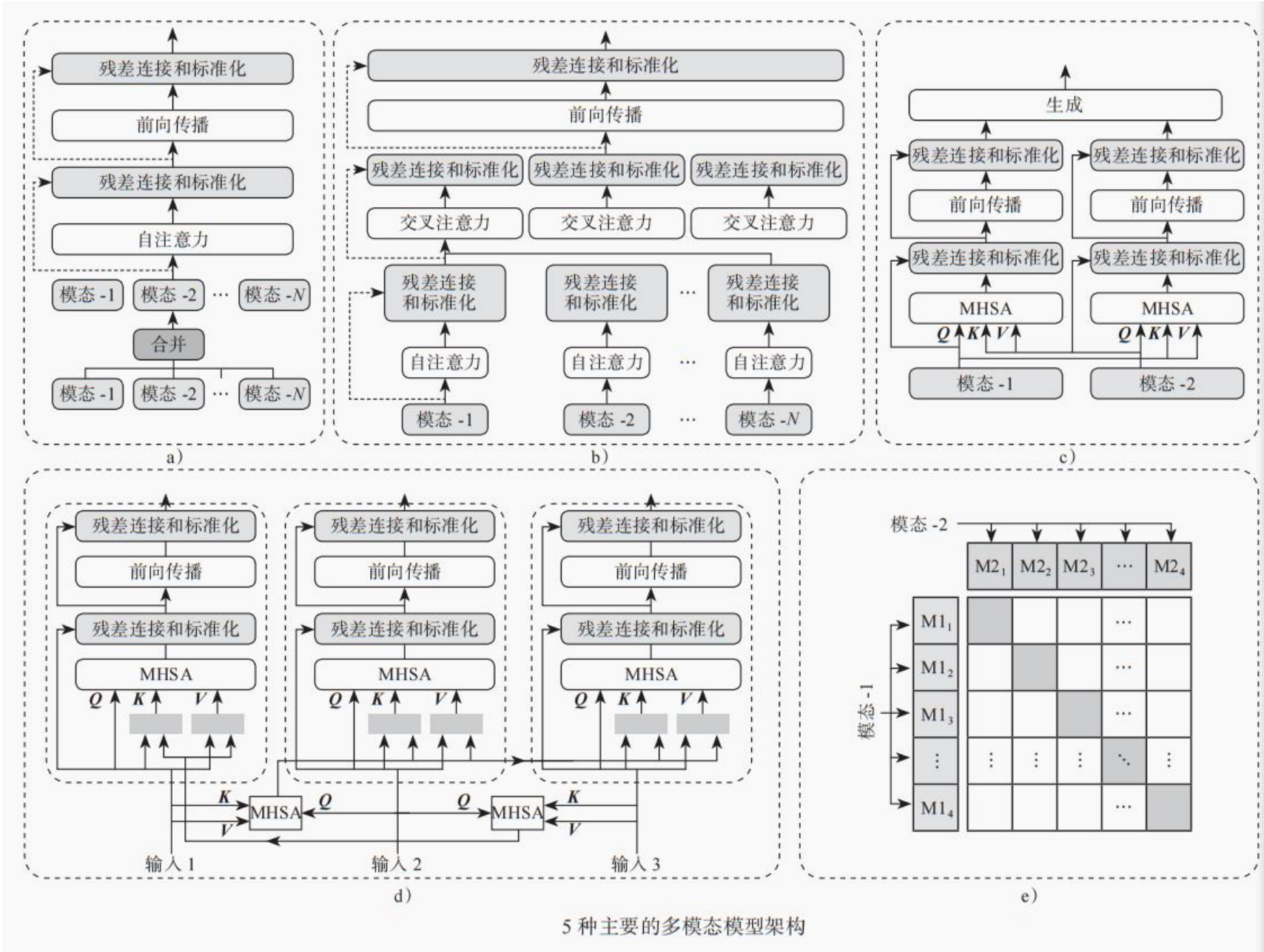
多模态模型能够从多种来源和模式中学习知识，并利用模式之间的交叉关联来完成任 务。例如，通过图像或图文知识库学习的信息可以用于回答自然语言问题；从文本中学习 的信息也可以应用于视觉任务。目前，多模态大模型正在经历将图文信息对齐、进行模态 认知管理、进一步形成多模态决策或生成的过程。常见的多模态大模型任务包括：

图像描 述生成或文本生成图像

(Stable Diffusion)、

图文问答 (GPT-4)、

从文本到图像或从图像到 文本的检索，以及视频流描述



Gemini和GPT-4V

GPT-4V

- GPT-4V是OpenAI的GPT-4模型的一个变体，其中的“V”代表“Vision”（视觉）。
- 这意味着GPT-4V增加了处理和理解图像的能力，使其能够分析图像内容并将其与文本信息相结合。
- GPT-4V可以用于各种视觉相关的任务，例如图像描述、视觉问答和图像分析。

•Gemini 是由 Google DeepMind 开发的大模型，最早于 2023 年 12 月 发布，目标是与 OpenAI 的 GPT-4 竞争。
Gemini 采用了多模态架构，可以处理文本、图像、音频、视频、代码等多种数据类型。

- 多模态能力：能同时理解文本、图像、代码，甚至音频和视频。
- 高效的推理能力：相比 GPT-4，在数学、科学推理和代码理解上表现更强。
- 轻量级版本：提供不同规模的模型，例如 Gemini Nano（轻量级）、Gemini Pro（主流版本）、Gemini Ultra。